



FUNDAMENTALS OF IOT AND SENSORS – 25EC1203

Experiment No - 2

Experiment No	2	Student ID	
Date		Student Name	

Controlling LED Interface with Arduino using Push Button

Aim/Objective: The main objective of this experiment is, to turn ON an LED when a button is pressed, OFF when button is released.

Components required:

Software	Hardware
Arduino IDE (To be installed in Laptop)	Arduino UNO Board Arduino UNO Cable LED (Red-1) Push Button - 1 Resistors – 2 (1K Ω each) Jumper wires

Pre-Requisites:

- Electronics fundamentals
- Basic knowledge about Arduino Pins

Pre-Lab:

1. What is the function of push button in Arduino?

Pushbuttons or switches connect two points in a circuit. When the button is pressed, LED is ON and when the button is released LED is off.

2. What is the use of LED in Arduino?

LED (Light Emitting Diode) is an electronic device, which emits light when the current passes through its terminals. It is used as an ON/OFF indicator in Arduino.

3. What is the value of the push button in Arduino?

When a push button is plugged to a digital pin, the value the Arduino reads is between 0V and 5V.

4. What are the types of push button?

- Single pole
- Single throw (SPST)
- Single pole
- Double throw (SPDT)
- Double pole, Single throw (DPST)
- Double pole, double throw (DPDT)

5. How many pins are there for push button?

Push buttons have four different pins, with two on each side. The pins that are across from each other are internally connected. This means that if one wire is connected to the pin on the top left and another to the top right pin the two wires would be connected together.

In-Lab:

PROGRAM:

Turn ON an LED when a button is pressed, OFF when button is released.

```
// Define the pin to which the button is connected
int buttonPin = 7;
// Define the pin to which the LED is connected
int ledPin = 8;

void setup()
{
  Serial.begin(9600);
  // Set pin 7 as an INPUT (for the button)
  pinMode(buttonPin, INPUT);
  // Set pin 8 as an OUTPUT (for the LED)
  pinMode(ledPin, OUTPUT);
}

void loop()
{
  // Read the state of the button and store it in the variable stateButton
  int stateButton = digitalRead(buttonPin);
  Serial.print("Status of the button is::");
  Serial.println(stateButton);

  // Make a decision based on the button state
  if (stateButton == HIGH)
  {
    // Button is pressed, provide voltage to pin 8 (turn on the LED)
    digitalWrite(ledPin, HIGH);
  }

  else
  {
    // Button is not pressed, do not output voltage on pin 8 (turn off the LED)
    digitalWrite(ledPin, LOW);
  }
}
```

PROCEDURE:

I. Connect the Components:

- Place the Arduino board on a stable surface.
- Connect one leg of the push-button switch to digital pin 7 on the Arduino.
- Connect the other leg of the push-button switch to one end of a resistor.
- Connect the other end of the resistor to the ground (GND) pin on the Arduino to create a pull-down resistor configuration.
- Connect one leg of the LED (the longer leg, the anode) to digital pin 8 on the Arduino.
- Connect the shorter leg of the LED (the cathode) to a current-limiting resistor (220-330 ohms).
- Connect the other end of the resistor to the ground (GND) pin on the Arduino to complete the LED circuit.

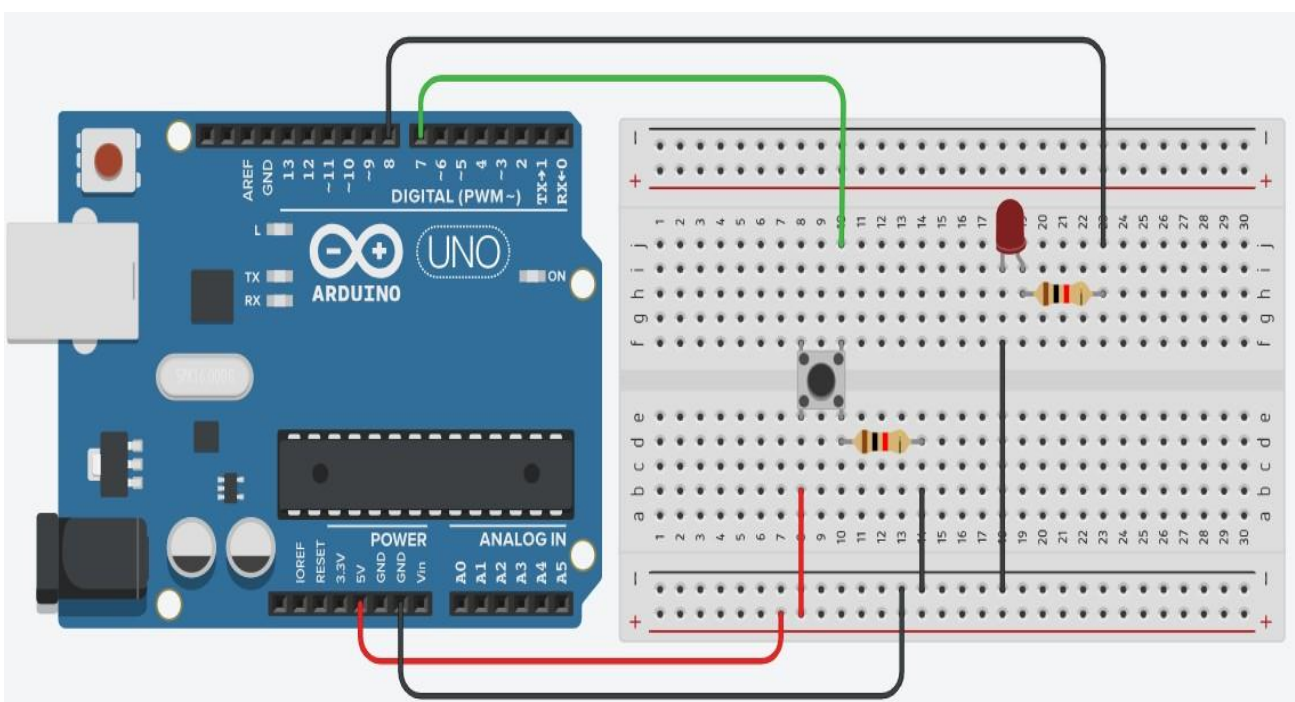
II. Upload the Arduino Code:

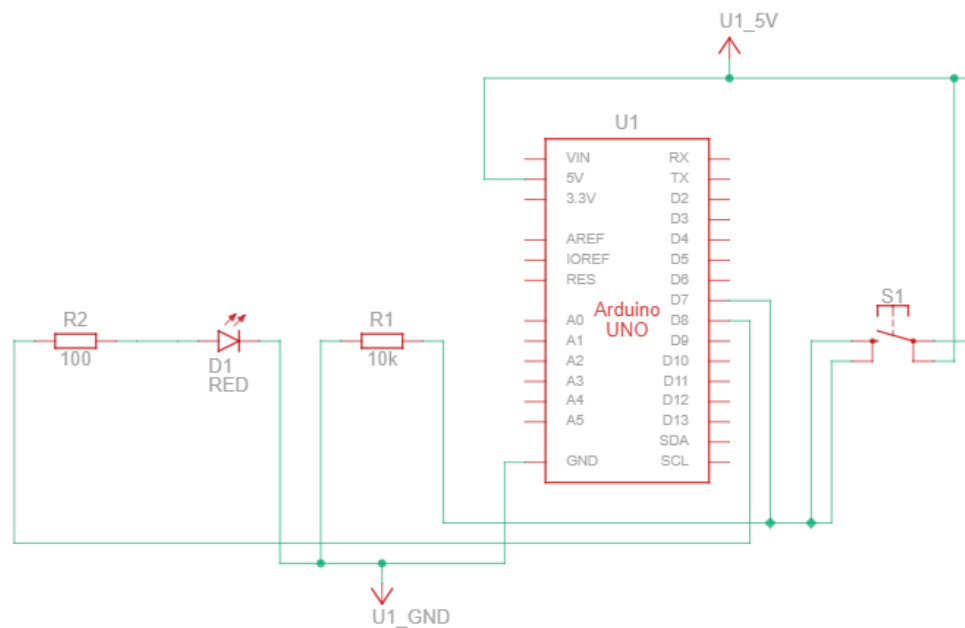
- Open the Arduino IDE on your computer.
- Create a new sketch and write the code provided into the sketch.
- Verify the code for any errors by clicking on the checkmark icon.
- If the code verifies successfully, upload it to your Arduino board by clicking on the right arrow icon.

III. Test the Circuit:

- After uploading the code, open the Serial Monitor in the Arduino IDE to observe the button's state.
- When you press the button, the LED connected to pin 8 should turn on. When you release the button, the LED should turn off.

Connection Diagram:





Results:

VIVA-VOCE Questions (In-Lab):

1. What is meant by button debouncing?

Push button is physically “bouncing” when it is touched (going from LOW to HIGH and HIGH to LOW many times in a short interval). This might be due to some error in the code.

2. How does push button work in Arduino?

When the button is closed (pressed), it makes a connection between its two legs, connecting the pin to 5 volts, so that we read a HIGH and when the button is released, the pin is connected to ground to read a LOW.

3. What is the push button to reset Arduino?

The RESET button is a white or blue push button located on top of your Arduino board. Pressing it has the same effect as disconnecting and reconnecting the power supply.

4. Does push button need resistor?

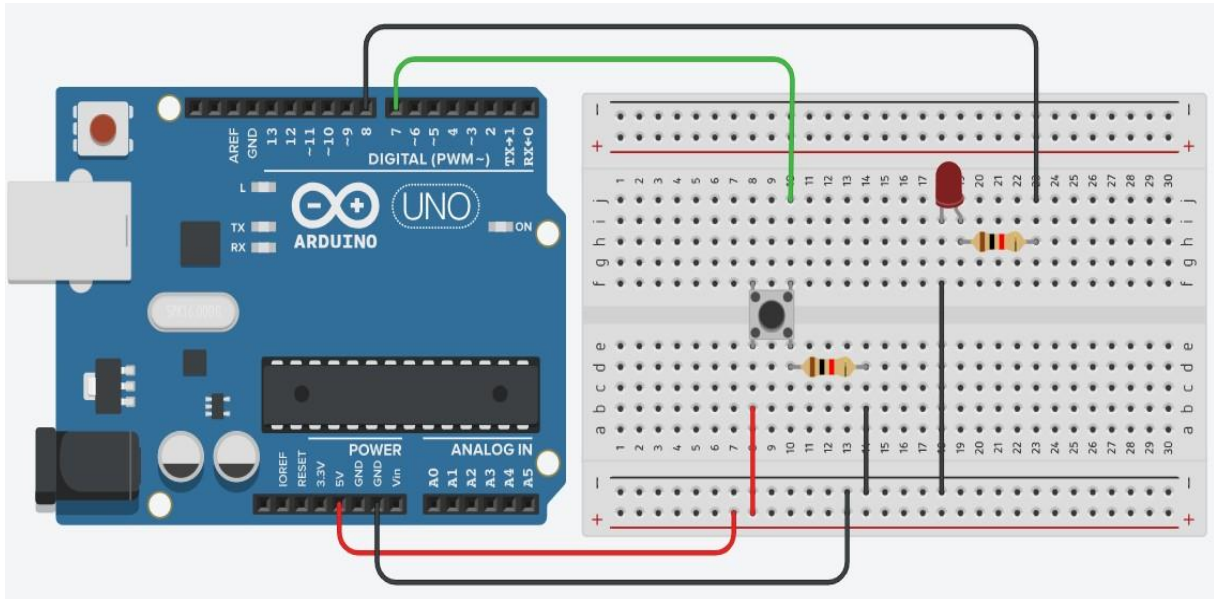
The resistor is mandatory for proper operation of a button.

5. What are the applications of push button?

Push button switches are used in a wide range of applications, including computers, crosswalks, telephones, industrial machinery, security systems, ATMs, military equipment, casino gambling slot machines, fitness equipment, and gadgets.

Post-Lab:

1. Write a program to blink an LED with a delay of one second when the push button released, LED off when button pressed.



Program:

// Constants

const int ledPin = 8; // LED connected to digital pin 8

const int buttonPin = 7; // Push button connected to digital pin 7

// Variables

int buttonState = 0; // Variable for reading the push button status

bool ledState = LOW; // Variable for storing the LED state

void setup() {

 // Initialize the LED pin as an output:
 pinMode(ledPin, OUTPUT);

 // Initialize the push button pin as an input:
 pinMode(buttonPin, INPUT_PULLUP); // Use internal pull-up resistor
}

void loop() {

 // Read the state of the push button value:
 buttonState = digitalRead(buttonPin);

 // Check if the push button is pressed
 if (buttonState == LOW) { // LOW means the button is pressed
 ledState = !ledState; // Toggle the LED state
 digitalWrite(ledPin, ledState); // Set the LED with the new state
 delay(1000); // Wait for one second
 }

}